

DeepPoint: MultiModal ISAC Sensing for Search-and-Rescue Robotics

ECSE 6660 Internetworking of Things

Final Report - April 2026

Adithya Vijayan, vijaya3@rpi.edu, ECSE 6660

Swastik Kanjilal, kanjis2@rpi.edu, ECSE 6660

1. Objective

Search-and-rescue is a dangerous guessing game. Firefighters and rescue teams lose visibility the moment they enter a burning building or a collapsed structure. Cameras fail in smoke, and LiDAR is scattered by dense particulate. The core research question driving DeepPoint is: can a deep learning super-resolution pipeline, trained on one mmWave radar platform, be deployed on a different, lower-cost radar mounted on a mobile robot to produce LiDAR-quality 3D point clouds in visually degraded environments?

DeepPoint reframes the limited resolution of compact mmWave radar as a decoding problem rather than a hardware limitation, applying a learned super-resolution model to recover dense, LiDAR-like point clouds from sparse radar input. This enables ground robots — and, in future work, drones — to map debris fields and track victims when optical sensors are blinded. The project also serves as an IoT class demonstration of multimodal sensor fusion, end-to-end deep learning deployment on a mobile platform, and lightweight cryptographic protection of mission-critical sensor data.

2. Background and Related Work

Learned super-resolution of mmWave radar is an active research area, and our project builds directly on existing work rather than introducing a new algorithmic contribution. The most relevant prior work is summarized in Table 1; the remainder of this section locates our project within it.

Prior Work	Relevance to DeepPoint
RadarHD (Prabhakara et al., ICRA '23) [1]	Direct foundation. Introduces the asymmetric U-Net architecture, 40-frame temporal stacking, low-threshold polar preprocessing, and weighted BCE + Dice loss that DeepPoint uses. Trained and evaluated on a TI AWR 1843 cascaded radar in static indoor environments.
Radarize (Sie et al., MobiSys '24) [2]	Uses Doppler shift directly for radar odometry, achieving lower drift than IMU-reliant methods in featureless hallways. Identified as a natural next step for live SLAM integration in our future work.
Cross-Modal Contrastive Learning for mmWave Navigation (Huang et al., RA-L '21) [3]	Demonstrates that mmWave radar can support navigation in adverse conditions (smoke) when trained against optical ground truth, motivating the broader case for radar-only deployment after multi-modal training.
U-Net (Ronneberger et al., MICCAI '15) [4] and V-Net (Milletari et al., 3DV '16) [5]	Provide the encoder–decoder backbone and the Dice loss that RadarHD — and by extension our project — adapts from medical image segmentation.

Table 1: Prior work that DeepPoint builds on.

RadarHD [1] is the immediate foundation of our project. It establishes the asymmetric U-Net architecture, the 40-frame temporal input representation, and the loss-function design that we adopt without modification. Our project is therefore best characterized as an applied extension of RadarHD rather than a new method. Within that scope, the contributions of this work are:

- An end-to-end reproduction of the RadarHD training pipeline on the RPI AiMOS HPC cluster, demonstrating that the published architecture and loss formulation converge stably on independent compute infrastructure.
- A custom synchronized multi-sensor dataset collected on a TurtleBot 4 platform, comprising paired Analog Devices 24 GHz radar, LiDAR, and depth-camera frames.
- An empirical evaluation of cross-platform transfer: applying a model trained on a 77 GHz TI cascaded radar to a 24 GHz Analog Devices single-chip radar, and reporting where the trained model succeeds and fails on this out-of-distribution sensor.
- An RSA proof-of-concept implementation that protects archived sensor payloads end-to-end, demonstrating the cryptographic workflow envisioned for a future operator-to-robot communication link.

We do not claim a new super-resolution algorithm. The value of the project lies in the systems-level integration — collecting in-house data with a different radar than the one RadarHD was developed for, evaluating cross-sensor generalization, and adding a cryptographic layer absent from prior radar super-resolution work.

3. Methodology

Our pipeline has three components: a deep learning super-resolution network that maps low-resolution radar heatmaps to LiDAR-like point clouds; a custom multi-sensor dataset collected on a TurtleBot 4 platform; and a cryptographic proof-of-concept that demonstrates end-to-end protection of serialized sensor archives. The overall block diagram is shown in Figure 1.

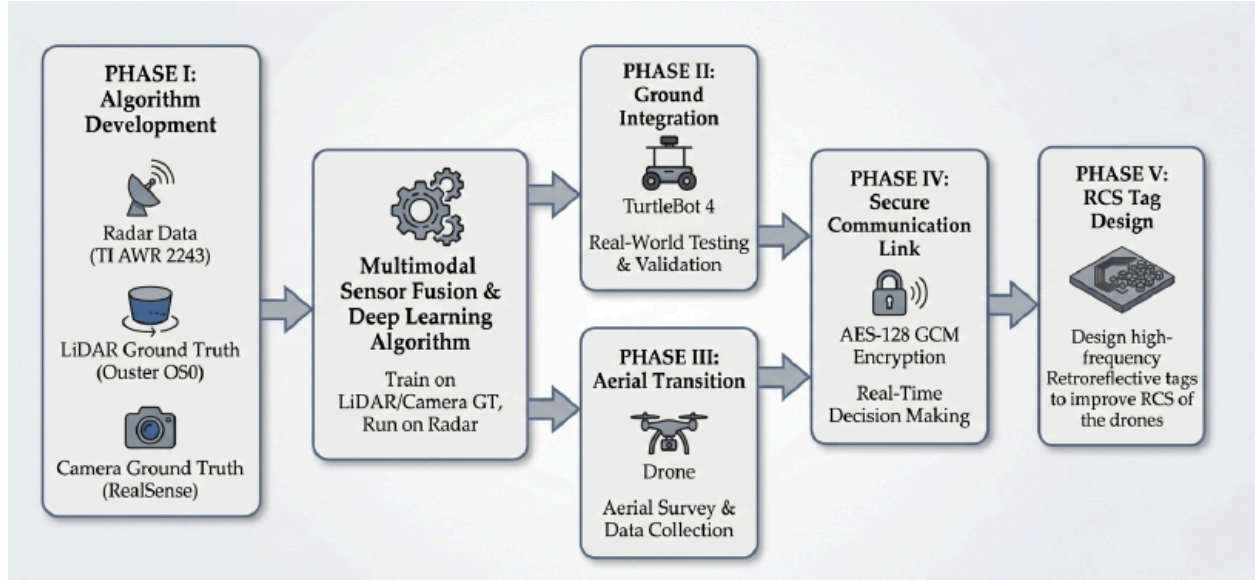


Figure 1: DeepPoint system block diagram showing the three implemented components.

3.1 Super-Resolution Architecture

The core of DeepPoint's super-resolution pipeline is an asymmetric U-Net encoder-decoder, adapted from RadarHD [1]. The encoder progressively downsamples the input through convolutional layers with increasing filter depth, building a global understanding of the scene that distinguishes true reflectors from sidelobe artifacts. The decoder upsamples back to a high-resolution range-azimuth polar image. Unlike a standard symmetric U-Net, the decoder is wider than the encoder because the super-resolution task is asymmetric: the output azimuthal width (512 pixels) is $8\times$ the input (64 pixels). Skip connections preserve fine spatial detail that downsampling would otherwise discard.

A critical design choice is stacking 40 past radar frames as input channels (~ 2 seconds of history at 20 Hz). This provides multiple viewpoints of the scene as the radar moves, mitigating specular-induced ghost points and smoothing temporal jitter. The network input is a $41 \times 256 \times 64$ tensor in polar coordinates. Polar-to-Cartesian conversion is performed only after inference, preserving the radial/azimuthal structure the U-Net exploits.

Training uses a weighted combination of Binary Cross-Entropy and Dice loss ($w_{BCE} = 0.9$, $w_{Dice} = 0.1$) with the Adam optimizer at a learning rate of 1×10^{-4} . BCE handles per-pixel matching against the LiDAR ground truth, while Dice loss promotes overlap between predicted and true occupied regions and compensates for the severe class imbalance between occupied and unoccupied pixels.

3.2 Custom Dataset Collection

A central deliverable of this project is a synchronized multi-sensor dataset collected on a TurtleBot 4 ground robot in indoor environments representative of search-and-rescue conditions. The platform integrates three sensors: an Analog Devices 24 GHz mmWave radar (a lower-cost, lower-frequency platform than the TI AWR 1843 used in the RadarHD reference dataset), a LiDAR providing ground truth, and an Intel RealSense depth camera providing complementary geometric ground truth. The hardware mounting is shown in Figure 2.

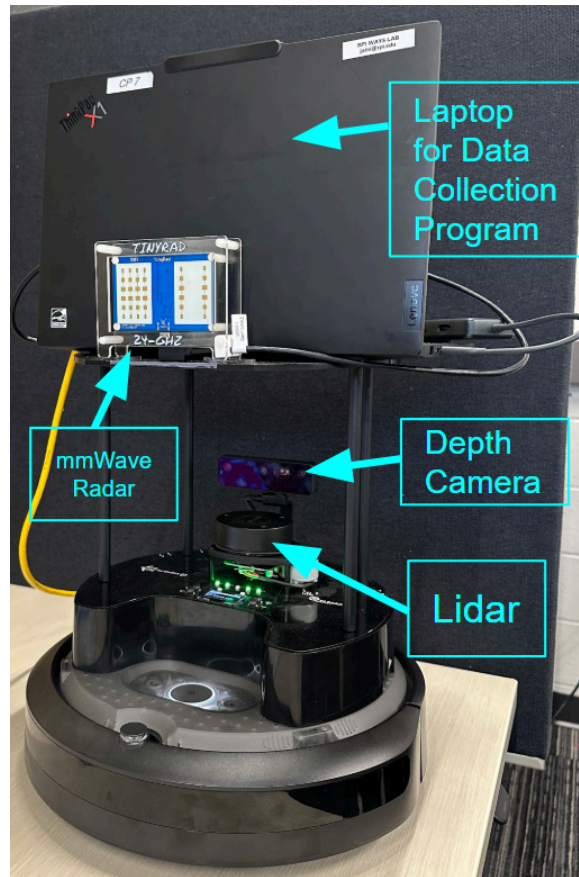


Figure 2: TurtleBot 4 sensor stack used for custom dataset collection.

The choice of the 24 GHz Analog Devices radar — rather than the 77 GHz TI AWR-series radar used by RadarHD — is deliberate. It tests cross-platform generalization: whether a model trained on one radar architecture can produce useful output on a sensor with different center frequency, antenna geometry, and angular resolution. Range and azimuth bins are reformatted into the polar input grid expected by the trained U-Net. Forward-facing LiDAR points within ± 30 cm elevation are retained and projected into the same polar coordinate frame to serve as evaluation ground truth.

3.3 Evaluation Protocol

Quantitative evaluation against the RadarHD test set uses two standard point-cloud metrics. Chamfer distance measures the average bidirectional nearest-neighbor distance between the predicted and ground-truth point clouds. Modified Hausdorff distance is more robust to outliers than the standard Hausdorff distance and is widely used in the radar SLAM literature. Qualitative evaluation on our custom

TurtleBot dataset compares the U-Net output side-by-side with the raw radar heatmap and the LiDAR ground truth, allowing inspection of cross-platform generalization.

3.4 Security Proof-of-Concept

We implemented RSA encryption as a proof-of-concept to protect our sensor data. RSA works by generating two keys: a public key to encrypt data and a private key to decrypt it. We took a ZIP file of our collected sensor frames, broke it into small chunks, encrypted each chunk, and saved it to a JSON file. To verify it worked, we decrypted it back and confirmed the recovered file was identical to the original. We did the same using the cryptography library in python for the AES comparison portion.

4. Hardware/Software Used

Table 2 summarizes the hardware and software stack used to develop, train, and evaluate DeepPoint.

Component	Role
Analog Devices 24 GHz mmWave radar	Primary low-resolution radar sensor on TurtleBot
LiDAR	Ground-truth point cloud for training and evaluation
Depth Camera	Complementary geometric ground truth
TurtleBot 4 (ground platform)	Mobile sensor platform for dataset collection
RPI AiMOS HPC cluster (Slurm GPU nodes)	U-Net training and hyperparameter sweeps
PyTorch	U-Net architecture and training
ROS 2	Sensor synchronization, robot control, data recording
MATLAB	Chamfer and modified Hausdorff distance computation
Custom Number_Package (course-developed)	RSA primitives: prime search, modular inverse, modular exponentiation
Python (standard library: json, pathlib, random)	Encrypt/decrypt orchestration and payload serialization

Table 2: Hardware and software components used in DeepPoint.

5. Final Results and Demo

5.1 Training Convergence

The U-Net was trained for 90 epochs on the RadarHD dataset using the AiMOS GPU nodes with the $wBCE = 0.9 / wDice = 0.1$ loss combination. Figure 3 shows the training loss converging smoothly from an initial value above 0.7 to a stable plateau near 0.07, confirming that the network learned a meaningful radar-to-LiDAR mapping. The lack of late-epoch divergence indicates that the chosen learning rate and loss weighting did not over-fit to fine LiDAR detail.

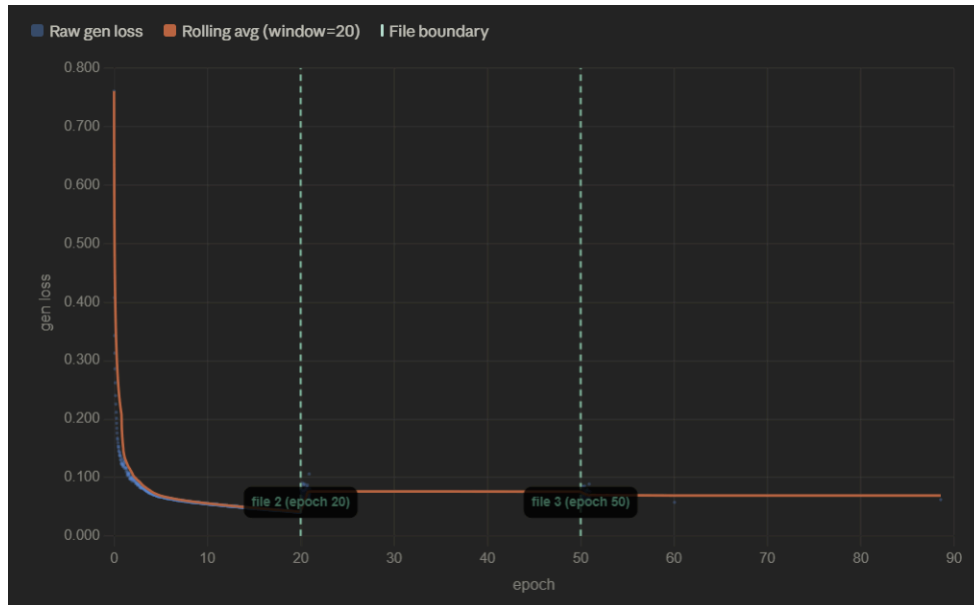


Figure 3: Weighted BCE + Dice training loss converging over 90 epochs on the AiMOS HPC cluster.

5.2 Qualitative Results on Custom TurtleBot Dataset

Figure 4 shows representative side-by-side outputs from our custom dataset: here we can see what the robot actually “sees” when we are collecting data. As we can see in figure 4 the LiDAR and depth camera provide very high resolution compared to the Radar, which is very noisy. However, when we pass it through the ML model, we can see that it can pull out some interesting features, as shown in figure 5. This shows that using this type of architecture, and exploring beyond, it can definitely be a solid way of completing our task.

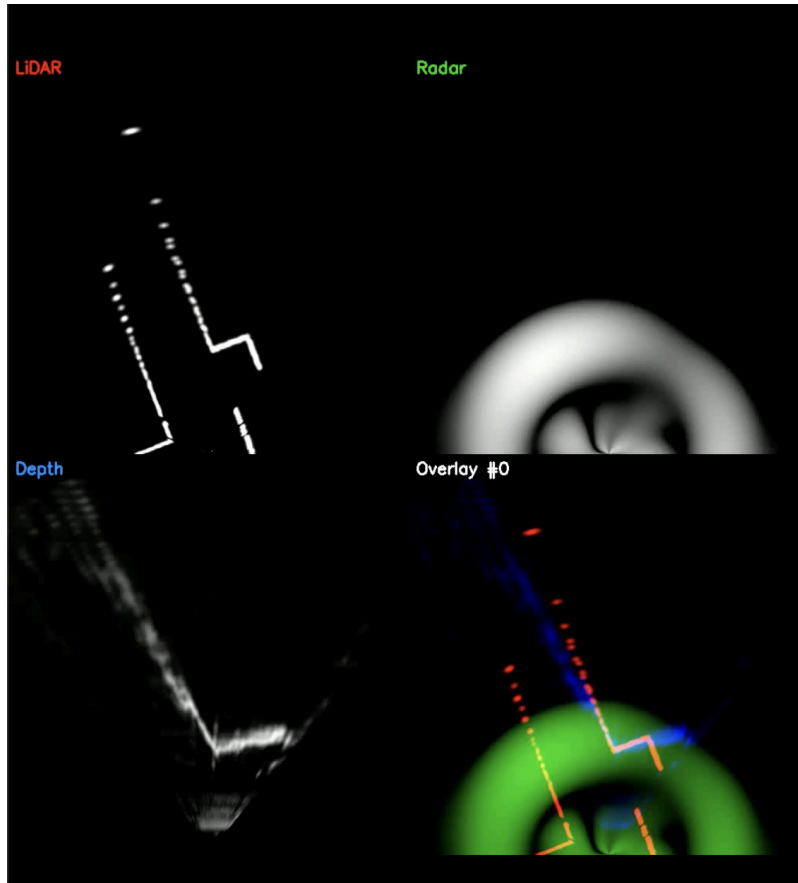


Figure 4: Qualitative comparison on a representative frame from the custom TurtleBot dataset.



Figure 5: Here is our output from the ML model showing that some of the extracted radar features

5.3 Security PoC Result

A 1.2 MB ZIP archive containing selected depth-camera frames was encrypted using the implemented RSA scheme, serialized to a JSON ciphertext file, and decrypted on the same host. The recovered ZIP was verified to be byte-identical to the original. This was done as a proof of concept, using 40 of our images. If we extrapolate to our 2600 images, we can see how long that would take in terms of RSA.

	RSA	AES
Encryption Timing	1243.885 ms	28.100 ms
Decryption Timing	1763.812 ms	70.728 ms

Table 1: This table compares the time it takes to encrypt and decrypt using both RSA and AES

5.4 Live Demo

The end-of-term demonstration walks through the full pipeline end-to-end on a recorded TurtleBot run: raw radar I/Q from the Analog Devices sensor → polar heatmap → U-Net super-resolved output → LiDAR-like point cloud, displayed alongside the simultaneously recorded LiDAR ground truth for visual comparison (figure 5). The demo concludes with the RSA proof-of-concept: a live encryption and decryption round-trip on a sample frame archive, with verified byte-identical recovery (table 1).

6. Conclusion

DeepPoint demonstrates that a learned super-resolution model can be trained end-to-end to map low-resolution mmWave radar to LiDAR-quality point clouds, and — importantly — that the trained model produces structurally meaningful output when deployed on a different, lower-cost radar mounted on a mobile robot. Our concrete deliverables are: a RadarHD-based U-Net implementation; a synchronized multi-sensor dataset collected on a TurtleBot 4 with an Analog Devices 24 GHz radar, an LiDAR, and a depth camera; quantitative and qualitative evaluation including a cross-platform generalization analysis; and a working RSA cryptographic proof-of-concept that protects archived sensor payloads end-to-end.

Limitations. The training-to-deployment domain gap (77 GHz TI radar → 24 GHz Analog Devices radar) is significant and visibly affects qualitative output quality. The model was not fine-tuned on in-domain data due to dataset size constraints..

Future Work. Three concrete extensions follow directly from the limitations above. (1) Fine-tune the U-Net on a modest amount of in-domain data from the Analog Devices radar; this is far cheaper than full retraining and is the most likely path to closing the cross-platform gap. (2) Integrate the pipeline with Doppler-based odometry [2] and a SLAM backend (Google Cartographer) for live, radar-only mapping in visually degraded environments, and ultimately port the stack to an aerial platform for rubble-field surveys.

7. Acknowledgements

We utilized the open-source repository *RadarHD* (<https://github.com/akarsh-prabhakara/RadarHD>) as a baseline framework for radar-based perception. The repository provided the core model architecture used in this project.

We collected our own dataset using a TurtleBot-based robotic platform equipped with LiDAR, a depth camera, and an Analog Devices 24 GHz radar. We adapted the pipeline to process our data, primarily by integrating our sensing setup and ensuring compatibility with the existing framework. All experimental results are based on our collected data and system implementation.

We would also like to acknowledge Tony Yu and Hyunsuh Eun (from our Ways Lab) for support in the ML training and data collection with the TurtleBot4.

8. References

- [1] A. Prabhakara, T. Jin, R. Das, G. Srivatsav, S. Kumar, C. Ruiz, and S. Kumar, "RadarHD: High Resolution Point Clouds from Low Resolution Radar," in Proc. IEEE Int. Conf. Robotics and Automation (ICRA), London, UK, 2023.
- [2] E. Sie, X. Wu, H. Guo, and D. Vasisht, "Radarize: Enhancing Radar SLAM with Generalizable Doppler-Based Odometry," in Proc. ACM MobiSys, Tokyo, Japan, 2024.
- [3] J.-T. Huang, C.-L. Lu, P.-K. Chang, C.-I. Huang, C.-C. Hsu, Z. L. Ewe, P.-J. Huang, and H.-C. Wang, "Cross-Modal Contrastive Learning of Representations for Navigation Using Lightweight, Low-Cost Millimeter Wave Radar for Adverse Environmental Conditions," IEEE Robotics and Automation Letters, vol. 6, no. 2, pp. 3333–3340, Apr. 2021.
- [4] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in Proc. MICCAI, Munich, Germany, 2015.
- [5] V. Milletari, N. Navab, and S.-A. Ahmadi, "V-Net: Fully Convolutional Neural Networks for Volumetric Medical Image Segmentation," in Proc. 3DV, Stanford, CA, 2016.
- [6] Analog Devices, "24 GHz mmWave Radar Sensor — Datasheet," 2024.